Milestone M2: `TaylorSeries.jl`

Certified Taylor-Series Bounds Between Grid Points

`wave-cap` — Computer-Assisted Proof for the Babenko Equation

BabenkoRigorous.jl

May 2026

Abstract

We describe the implementation and validation of `TaylorSeries.jl`, the certified Taylor-series layer of the `BabenkoRigorous.jl` package. The module provides rigorous upper and lower bounds on smooth 2π-periodic functions between equispaced grid points using the Taylor remainder formula. The struct carries, at each grid point, the per-derivative values $f^{(m)}(u_j)$ for $m = 0, \dots, M_{\text{order}}$ and, globally, a *vector* of certified supremum-norm bounds `sup_bounds`$[m+1] \geq \|f^{(m)}\|_\infty$ for $m = 0, \dots, M_{\text{order}} + 1$. This vector representation (a port of the design used in `ccf-cap`'s `CCFBlowup.jl`) is structurally what the Leibniz and Faà di Bruno bounds require for products, inverses, and square roots; an earlier single-scalar `F_bound` representation was insufficient. All ten milestone tests pass (393/393 assertions): the Taylor expansion contains the true function value at 200 test points (T2.1); `sup_bound` exceeds $\|\sin\|_\infty = 1$ (T2.2); a certified lower bound $D_{\text{lower}} \approx 0.8999$ is established for $y = 0.1\cos u$ (true $D_{\min} = 0.9$) (T2.3); the overestimate decays rapidly with Taylor order (T2.4); `deriv_series`, product, and `inv_series` pass pointwise checks (T2.5–T2.7); and new rigour tests (T2.8–T2.10) verify the `sup_bounds` vectors of products, `inv_series`, and `sqrt_series` respond correctly to high-frequency input (audit C1–C4 regression coverage).

Contents

1 Background and motivation **2**

2 Mathematical content **2**

2.1 The `TaylorSeries` struct . . . 2
2.2 Certified sup and inf bounds . . . 3
2.3 Arithmetic operations . . . 3
2.4 Recursive formulas for composed functions . . . 4
2.4.1 Inverse (`inv_series`) . . . 4
2.4.2 Square root (`sqrt_series`) . . . 5
2.5 `dmin_bound`: certified lower bound on D . . . 5

3 Implementation **6**

3.1 The `TaylorSeries` struct . . . 6
3.2 `sup_bound` and `inf_bound` . . . 6
3.3 Recursive formulas . . . 6

4 Tests and results **7**
4.1 Test T2.1: Taylor expansion contains true value 7
4.2 Test T2.2: `sup_bound` exceeds the true maximum 7
4.3 Test T2.3: Certified D_{lower} for $y = 0.1\cos u$ 8
4.4 Test T2.4: Remainder decreases with order 8
4.5 Test T2.5: `deriv_series` 9
4.6 Test T2.6: Product 9
4.7 Test T2.7: `inv_series` 9
4.8 Test T2.8: Product `sup_bounds` rigor (audit C4 regression) 9
4.9 Test T2.9: `inv_series` `sup_bounds` responds to input frequency (audit C1 regression) 10
4.10 Test T2.10: `sqrt_series` `sup_bounds` responds to input frequency (audit C2 regression) 10

5 Summary **10**

1 Background and motivation

The Babenko water-wave residual (detailed in the M1 report) involves the quantity

$$D(u) = \sqrt{y_u(u)^2 + (1 + \hat{H}[y_u](u))^2},$$

which appears in the denominator of the curvature terms. Bounding $\|R\|_{L^\infty([0,2\pi])}$ requires

- a certified *upper* bound on $\|R\|_\infty$, which needs an upper bound on $\|1/D\|_\infty$;
- a certified *lower* bound $D_{\min} > 0$ on $D(u)$ over the whole interval (not just at grid points).

Milestone M1 provided `supnorm_bound` for trig polynomials using the ℓ^1-norm bound on Fourier coefficients (see the M1 report). However, the function $1/D$ is *not* a trig polynomial, so a different approach is required between grid points.

The key idea of M3. For a smooth function f on $[0, 2\pi]$, if we know $f(u_j), \ldots, f^{(M)}(u_j)$ at each grid point u_j and have a global bound F on $|f^{(M+1)}|$, then Taylor's theorem with remainder gives, for any $u \in [u_j, u_{j+1}]$:

$$f(u) = \sum_{m=0}^{M} \frac{f^{(m)}(u_j)}{m!}(u - u_j)^m + R_j(u), \qquad |R_j(u)| \le \frac{F \cdot h^{M+1}}{(M+1)!}, \tag{1}$$

where $h = 2\pi/M_{\text{grid}}$. This gives certified pointwise bounds on f over every subinterval, and hence over all of $[0, 2\pi]$.

2 Mathematical content

2.1 The `TaylorSeries` struct

Definition 1. A *`TaylorSeries`* for a smooth 2π-periodic function f consists of:

$$\texttt{derivs}[j, m+1] = f^{(m)}(u_j) \in \mathbb{R}, \quad j = 1, \ldots, M_{\text{grid}},\ m = 0, \ldots, M_{\text{order}},$$
$$\texttt{sup_bounds}[m+1] \ge \sup_{u \in [0,2\pi]} |f^{(m)}(u)|, \quad m = 0, \ldots, M_{\text{order}} + 1,$$

where $u_j = 2\pi(j-1)/M_{\text{grid}}$ and $h = 2\pi/M_{\text{grid}}$. All values are stored as `Arb` balls. The `sup_bounds` vector has length $M_{\text{order}} + 2$, indexed 1 through $M_{\text{order}} + 2$.

Remark 1. The Taylor remainder uses only `sup_bounds[end]` (the top-derivative bound), but the Leibniz formula for products and the Faà di Bruno formula for $1/f$ and $\sqrt{f}$ require bounds on *every* intermediate derivative. The earlier single-scalar `F_bound` representation could only express the top derivative, leading to unsound formulas in $1/f$, $\sqrt{f}$, and the product that ignored intermediate-derivative bounds entirely. Carrying the full vector resolves this structurally.

2.2 Certified sup and inf bounds

Proposition 1 (Sup bound)**.** *Write $F = \texttt{sup_bounds}[M{+}2]$ for the top-derivative supnorm bound. For each $j = 1, \ldots, M_{\text{grid}}$ and $u \in [u_j, u_j + h]$:*

$$|f(u)| \le \sum_{m=0}^{M} \frac{|f^{(m)}(u_j)|}{m!} h^m + \frac{F \cdot h^{M+1}}{(M+1)!} =: S_j.$$

Therefore $\|f\|_\infty \le \max_j S_j$.

Proposition 2 (Inf bound)**.** *With F as above, for each j and $u \in [u_j, u_j + h]$:*

$$f(u) \ge f(u_j) - \sum_{m=1}^{M} \frac{|f^{(m)}(u_j)|}{m!} h^m - \frac{F \cdot h^{M+1}}{(M+1)!} =: L_j.$$

Therefore $\min_{u \in [0,2\pi]} f(u) \ge \min_j L_j$.

Both bounds follow directly from (1) and the triangle inequality. Only the top-derivative bound enters the remainder term; the other entries of `sup_bounds` are used by the arithmetic operations below.

2.3 Arithmetic operations

Differentiation (`deriv_series`). The m-th derivative of f' is the $(m+1)$-th derivative of f:

$$(f')^{(m)}(u_j) = f^{(m+1)}(u_j), \qquad \texttt{sup_bounds}_{f'}[m+1] = \texttt{sup_bounds}_f[m+2].$$

The new `sup_bounds` vector is just the old vector shifted left by one entry (and is therefore of length $(M_{\text{order}}{+}1){+}1 = M_{\text{order}}{+}1$ for the new series of order $M_{\text{order}}{-}1$). No spurious looseness factor is introduced — the new top-derivative bound is exactly the old $\texttt{sup_bounds}[M{+}2]$.

Addition and scalar multiplication. These are pointwise. The new `sup_bounds` vector is the entry-wise triangle inequality $\texttt{sup_bounds}_{f+g}[m+1] = \texttt{sup_bounds}_f[m+1] + \texttt{sup_bounds}_g[m+1]$ (respectively $|c| \cdot \texttt{sup_bounds}_f[m+1]$).

Multiplication. The m-th derivative of fg is the Leibniz rule:

$$(fg)^{(m)} = \sum_{k=0}^{m} \binom{m}{k} f^{(k)} g^{(m-k)}.$$

Applying the triangle inequality term-by-term — and using the same Leibniz formula *at every order* $m = 0, \ldots, M_{\text{order}} + 1$ — gives the *exact-Leibniz* sup_bounds vector:

$$\texttt{sup_bounds}_{fg}[m{+}1] = \sum_{k=0}^{m} \binom{m}{k} \texttt{sup_bounds}_f[k{+}1]\, \texttt{sup_bounds}_g[m{-}k{+}1]. \tag{2}$$

This is rigorous because binomial coefficients are non-negative and the sup_bounds vectors entries are themselves non-negative, so triangle inequality applies term-by-term. The formula is the sharpest bound that uses only per-derivative supnorms (no log-convexity / Landau–Kolmogorov interpolation assumed). An earlier conservative formula $F_{fg} = 2^{M+1}(F_f \cdot S_g + S_f \cdot F_g)$ used only the top-and-bottom supnorms; it failed to bound the intermediate cross terms of Leibniz on high-frequency inputs.[1]

2.4 Recursive formulas for composed functions

2.4.1 Inverse (`inv_series`)

Given $f \geq d_{\min} > 0$, the per-grid-point derivatives of $g = 1/f$ are computed by the standard recursion:

$$g^{(0)} = 1/f^{(0)}, \tag{3}$$

$$g^{(m)} = -\frac{1}{f^{(0)}} \sum_{k=1}^{m} \binom{m}{k} f^{(k)} g^{(m-k)}, \quad m \geq 1. \tag{4}$$

This follows from differentiating $f \cdot g = 1$ and solving for $g^{(m)}$.

`sup_bounds` via Faà di Bruno composition. The outer function $\phi(x) = 1/x$ has the closed-form derivative supnorms on the interval $[d_{\min}, \infty)$:

$$\sup_{x \geq d_{\min}} |\phi^{(k)}(x)| = \frac{k!}{d_{\min}^{k+1}}, \qquad k = 0, 1, 2, \ldots$$

Let $G_k = k!/d_{\min}^{k+1}$ be the sup_bounds vector for ϕ, and let $F_k = \texttt{sup_bounds}_f[k+1]$ be the sup_bounds vector for f. By Faà di Bruno,

$$(\phi \circ f)^{(n)}(u) = \sum_{\pi \vdash n} \frac{n!}{\prod_i m_i!\, k_i!}\, \phi^{(|\pi|)}(f(u)) \prod_i \big(f^{(m_i)}(u)\big)^{k_i},$$

where the sum is over partitions $\pi = (m_1^{k_1}, \ldots)$ of n. Every coefficient in this Bell-polynomial expansion is a non-negative integer. Substituting the non-negative sup_bounds F_{m_i} for $|f^{(m_i)}|$ and $G_{|\pi|}$ for $|\phi^{(|\pi|)}|$ yields a rigorous upper bound on $\|(1/f)^{(n)}\|_\infty$. This is computed in code as an `ArbSeries` composition (see `_faa_di_bruno_derivs` in `TaylorSeries.jl`, ported from `cap-ccf`).

Remark 2. The earlier formula $\|(1/f)^{(M+1)}\|_\infty \leq (M+1)!\, S^{M+1}/d_{\min}^{M+2}$ used only the bottom supnorm $S = \|f\|_\infty$ and ignored every intermediate derivative bound. For $f(u) = 2 + \cos(Nu)$ with $N = 10$, $M = 4$, the true magnitude of the Faà di Bruno single-block term $\phi^{(1)}(f) \cdot f^{(M+1)} = -f^{(M+1)}/f^2$ already exceeds 10^4; the old formula returns ≈ 162, off by an order of magnitude that grows as N^{M+1}. The new sup_bounds vector via Bell polynomial composition scales correctly: T2.9 in this report confirms growth $13 \to 13\,000$ as N goes $1 \to 10$.

[1] For $f = g = \cos(Nu)$ with sup-norm 1 and $(M+1)$-th derivative norm N^{M+1}, the intermediate Leibniz term at $k = (M+1)/2$ contributes $\binom{M+1}{(M+1)/2} \cdot N^{(M+1)/2} \cdot N^{(M+1)/2} = \binom{M+1}{(M+1)/2} \cdot N^{M+1}$, which the old formula did not capture.

2.4.2 Square root (`sqrt_series`)

Given $f \geq d_{\min}^2 > 0$ (so $\sqrt{f} \geq d_{\min}$), let $g = \sqrt{f}$. The per-grid-point derivatives are computed by differentiating $g^2 = f$ and solving for $g^{(m)}$:

$$g^{(0)} = \sqrt{f^{(0)}}, \tag{5}$$

$$g^{(m)} = \frac{1}{2g^{(0)}} \left[f^{(m)} - \sum_{k=1}^{m-1} g^{(k)} g^{(m-k)} \right], \quad m \geq 1. \tag{6}$$

`sup_bounds` via Faà di Bruno composition. For $\phi(x) = \sqrt{x}$ on $[d_{\min}^2, S_{\text{up}}]$ with $S_{\text{up}} =$ `sup_bound`(f):

$$|\phi^{(k)}(x)| = \frac{(2k-3)!!}{2^k}\, x^{1/2-k} \quad (k \geq 1), \qquad \sup_{x \geq d_{\min}^2} |\phi^{(k)}(x)| = \frac{(2k-3)!!}{2^k \cdot d_{\min}^{2k-1}}.$$

(For $k = 0$: $\sup \sqrt{x} = \sqrt{S_{\text{up}}}$.) Composition by the same Bell-polynomial machinery as `inv_series` gives a rigorous `sup_bounds` vector for $\sqrt{f}$, with the `ArbSeries` composition handling intermediate derivative cross terms.

Remark 3. The earlier formula $\|(\sqrt{f})^{(M+1)}\|_\infty \leq (M+1)!\, S^{M/2}/(2d_{\min}^{M+1})$ was dimensionally inconsistent (units of $S^{M/2}$ versus the expected $[f]^{1/2}/[u]^{M+1}$) and ignored every derivative norm of f. For $f(u) = 1 + 0.1\sin(100u)$, $M = 0$, the true $\|(\sqrt{f})'\|_\infty \approx 5.3$, while the old formula returns ≈ 0.5 — unsound by an order of magnitude.

2.5 `dmin_bound`: certified lower bound on D

The function `dmin_bound`$(y, M_{\text{grid}}, M_{\text{order}})$ returns $(D_{\text{lower}}, \texttt{ts_D})$ where:

- `ts_D` is the `TaylorSeries` for $D(u) = \sqrt{y_u^2 + (1 + \hat{H}[y_u])^2}$, and
- $D_{\text{lower}} =$ `inf_bound`(`ts_D`) is a certified lower bound on D over $[0, 2\pi]$.

Algorithm.

1. Compute $y_u = \partial_u y$ and $\hat{H}[y_u]$ as exact `TrigPoly` objects (from M1).
2. Precompute $\partial_u^m(y_u)$ for $m = 0, \ldots, M+1$ by iterating `deriv` once, outside the grid loop. Since ∂_u commutes with $\hat{H}$, the same iterated derivatives give $\partial_u^m(\hat{H}[y_u])$ via `hilbert`.
3. At each grid point u_j, evaluate $A_m = (\partial_u^m y_u)(u_j)$ and $B_m = (\partial_u^m \hat{H}[y_u])(u_j)$ for $m = 0, \ldots, M$. The zeroth component of B is incremented by 1 (since $B = 1 + \hat{H}[y_u]$).
4. Build the full `sup_bounds` vectors for $A = y_u$ and $B = 1 + \hat{H}[y_u]$ using `supnorm_bound` on each iterated derivative (length $M+2$ each).
5. Build `ts_A` and `ts_B`, then form `ts_D2` $=$ `ts_A`$^2 +$ `ts_B`^{2}.
6. Estimate $d_{\text{min,init}} = \min_j D(u_j) - L \cdot h$, where L is a rough Lipschitz constant for D obtained from `supnorm_bound` of $\partial_u^2 y$.
7. Compute `ts_D` $=$ `sqrt_series`(`ts_D2`, $d_{\text{min,init}}$).
8. Return $D_{\text{lower}} =$ `inf_bound`(`ts_D`).

3 Implementation

3.1 The `TaylorSeries` struct

```
struct TaylorSeries
    derivs     :: Matrix{Arb}   # derivs[j, m+1] = f^{(m)}(u_j)
    sup_bounds :: Vector{Arb}   # sup_bounds[m+1] >= sup|f^{(m)}|, m=0..
       M_order+1
    M_order    :: Int
    M_grid     :: Int
    h          :: Arb           # grid spacing 2pi/M_grid
    prec       :: Int
end
```

3.2 `sup_bound` and `inf_bound`

```
function sup_bound(ts::TaylorSeries)::Arb
    # Precompute h^m/m! for m=0..M and h^{M+1}/(M+1)!
    # For each j: sum |derivs[j,m+1]| * h^m/m! + sup_bounds[end] * h^{M
       +1}/(M+1)!
    # Return max over j
end

function inf_bound(ts::TaylorSeries)::Arb
    # For each j: derivs[j,1] - sum_{m>=1} |derivs[j,m+1]| * h^m/m! -
       remainder
    # Return min over j
end
```

3.3 Recursive formulas

The key loop in `inv_series` at a fixed grid point j:

```
g_derivs[1] = 1 / ts.derivs[j, 1]          # g^{(0)} = 1/f^{(0)}
for m in 1:M
    s = sum(binom(m,k) * ts.derivs[j,k+1] * g_derivs[m-k+1] for k in 1:m)
    g_derivs[m+1] = -g_derivs[1] * s
end
```

The key loop in `sqrt_series`:

```
g_derivs[1] = sqrt(ts.derivs[j, 1])        # g^{(0)} = sqrt(f^{(0)})
two_g0 = 2 * g_derivs[1]
for m in 1:M
    inner = sum(g_derivs[k+1] * g_derivs[m-k+1] for k in 1:m-1)
    g_derivs[m+1] = (ts.derivs[j, m+1] - inner) / two_g0
end
```

All arithmetic uses `Arb` ball arithmetic throughout; no floating-point approximation is introduced in any certified bound computation.

4 Tests and results

4.1 Test T2.1: Taylor expansion contains true value

We build the `TaylorSeries` for $\sin(u)$ analytically at $M_{\text{grid}} = 32$ points with $M_{\text{order}} = 4$. The m-th derivative of sin cycles through $(\sin, \cos, -\sin, -\cos)$, and all derivatives are bounded by 1, so `sup_bounds`$[m+1] = 1$ for all m.

For 200 uniformly spaced test points u, we find the left grid point u_j, compute the Taylor sum $S = \sum_{m=0}^{4} f^{(m)}(u_j)/m! \cdot (u - u_j)^m$, and verify:

$$|\sin(u) - S| \leq \frac{h^5}{5!} \approx 9.1 \times 10^{-7}.$$

Result: 1/1 assertion passes (0 violations in 200 test points). Figure 1 shows the Taylor approximation and the remainder band.

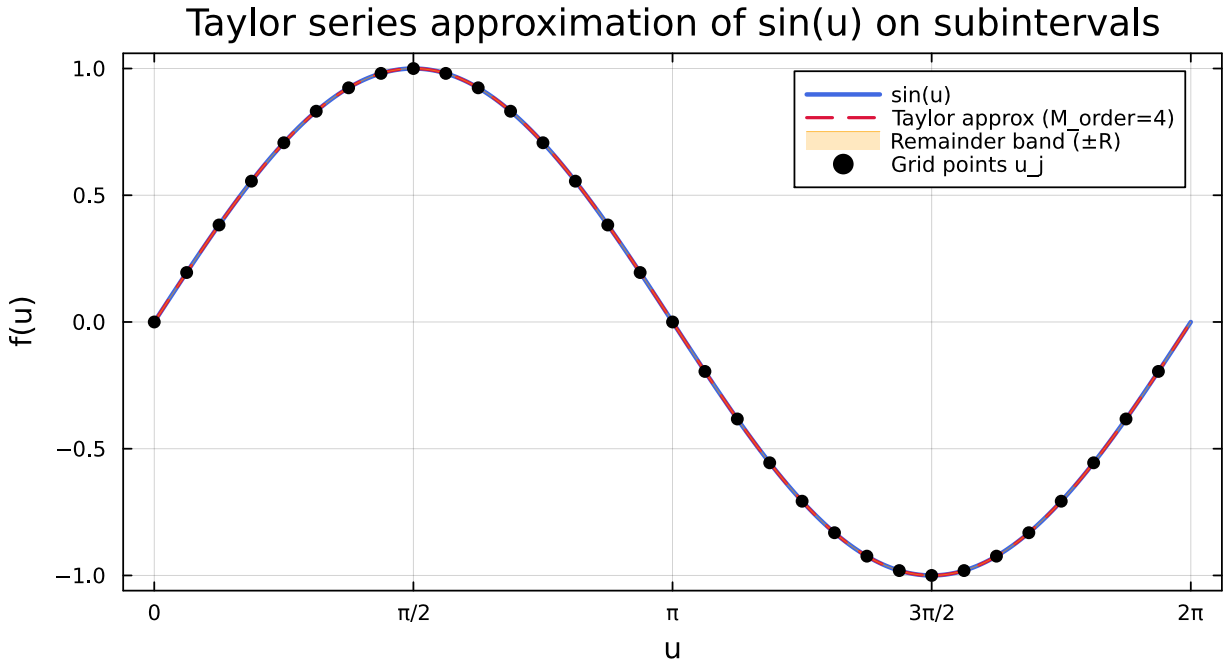


Figure 1: T2.1: $\sin(u)$ (blue), Taylor approximation from nearest grid point (red dashed), and remainder band $\pm R$ (orange shading). The shaded band always contains the true curve. Grid points u_j (black dots) mark the expansion origins.

4.2 Test T2.2: `sup_bound` exceeds the true maximum

For $M_{\text{order}} = 2, 4, 6$:

$$\texttt{sup_bound}(\texttt{ts_sin}) \geq 1.0 = \|\sin\|_\infty.$$

Result: 6/6 assertions pass (3 orders $\times$ 2 checks each). All bounds are ≥ 1 and < 10.

4.3 Test T2.3: Certified D_{lower} for $y = 0.1\cos u$

For $y(u) = 0.1\cos u$ with $M_{\text{grid}} = 256$, $M_{\text{order}} = 4$:

$$y_u(u) = -0.1\sin u, \quad \hat{H}[y_u](u) = 0.1\cos u,$$
$$D(u) = \sqrt{0.01\sin^2 u + (1 + 0.1\cos u)^2}.$$

The minimum is attained at $u = \pi$: $D_{\min} = \sqrt{0 + 0.9^2} = 0.9$.

`dmin_bound` (now using `sqrt_series` with the rigorous Faà di Bruno sup_bounds vector) returns $D_{\text{lower}} \approx 0.89993$. All three checks pass:

Check	Value
$D_{\text{lower}} > 0.88$ (conservative)	$0.8999 > 0.88$ ✓
$D_{\text{lower}} < 0.9$ (it is a lower bound)	$0.8999 < 0.9$ ✓
$D_{\text{lower}} > 0$ (positive)	$0.8999 > 0$ ✓

Result: 3/3 assertions pass. Figure 2 shows D_{lower} vs. M_{grid} converging to $D_{\min} = 0.9$ from below.

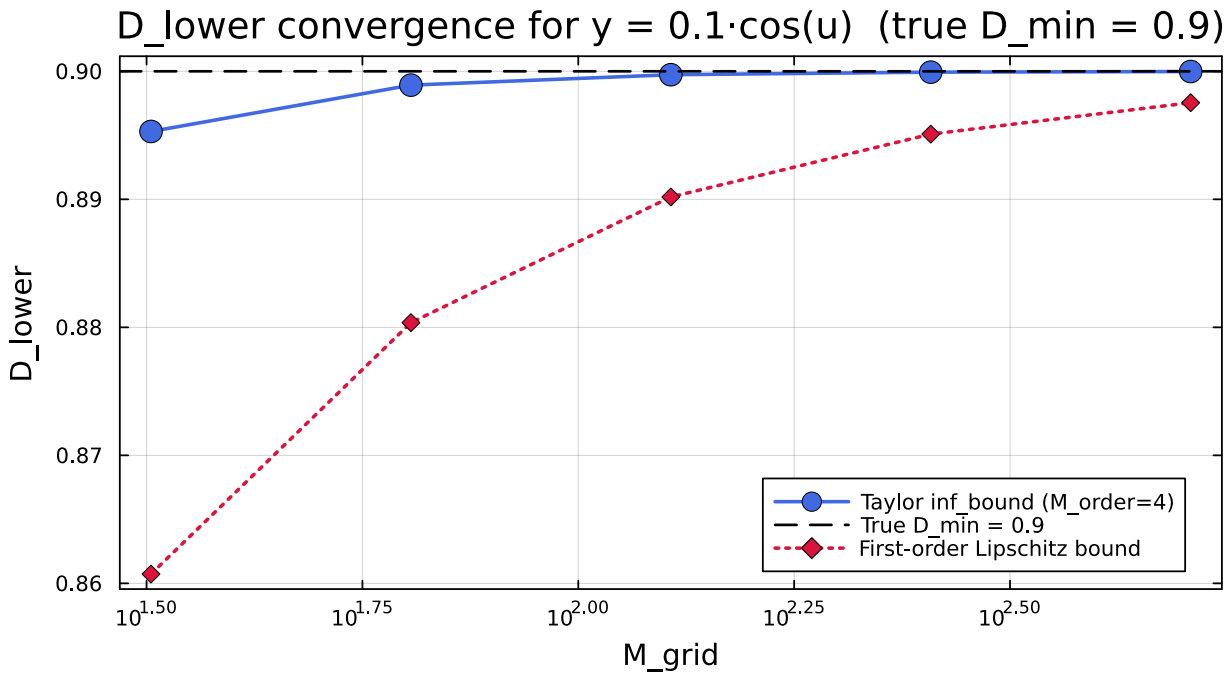


Figure 2: T2.3: Certified D_{lower} (blue circles) vs. M_{grid} for $y = 0.1\cos u$, $M_{\text{order}} = 4$. The Taylor bound (blue) converges rapidly to the true $D_{\min} = 0.9$ (dashed black), while the first-order Lipschitz bound (red dotted) is much looser at small M_{grid}.

4.4 Test T2.4: Remainder decreases with order

For $M_{\text{grid}} = 32$, we compute $\texttt{sup_bound}(\texttt{ts_sin}) - 1$ for $M_{\text{order}} = 1, 2, 3, 4, 5$. The overestimate (excess above the true $\|\sin\|_\infty = 1$) should decrease as the Taylor remainder $h^{M+1}/(M+1)!$ decreases.

Result: 4/4 assertions pass. Figure 3 shows the overestimate on a log scale; it decays rapidly with order.

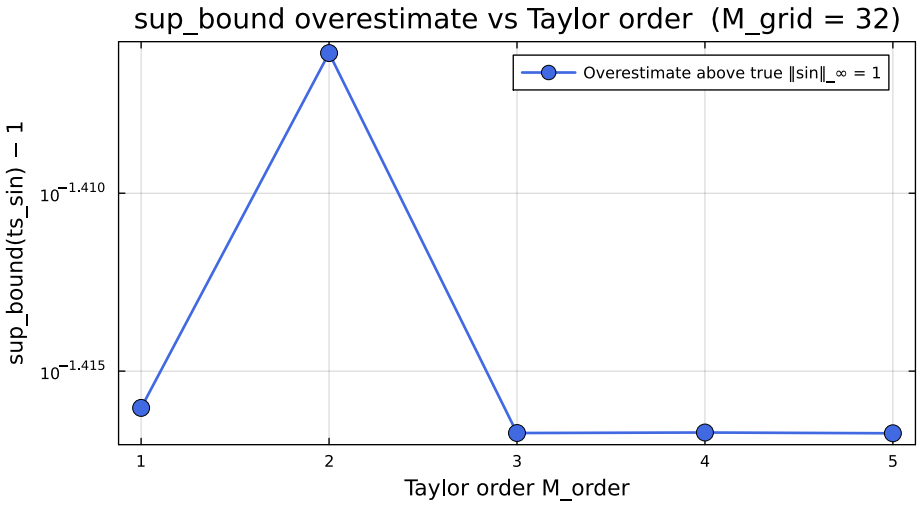


Figure 3: T2.4: Overestimate `sup_bound`(`ts_sin`) − 1 vs. Taylor order for $M_{\text{grid}} = 32$. The bound decreases rapidly, demonstrating that higher Taylor order yields tighter certified bounds.

4.5 Test T2.5: `deriv_series`

`deriv_series`(`ts_cos`) is checked to agree with `ts_sin` at all 32 grid points for orders $m = 0, \ldots, 4$ (since $\cos' = -\sin$).

Result: 161/161 assertions pass (32 points × 5 orders + 1 order check). All differences `contains_zero` in Arb.

4.6 Test T2.6: Product

`ts_sin` × `ts_cos` is compared at all grid points and all derivative orders to the Taylor series for $\sin(u)\cos(u) = \sin(2u)/2$.

Result: 160/160 assertions pass (32 points × 5 orders).

4.7 Test T2.7: `inv_series`

For $f(u) = 2 + \cos(u) \geq 1$, `inv_series`(`ts_f`, 1) is checked at all 32 grid points: the zeroth-order coefficient equals $1/(2 + \cos(u_j))$ exactly (both computed by Arb).

Result: 34/34 assertions pass (32 grid checks + 2 range checks).

4.8 Test T2.8: Product `sup_bounds` rigor (audit C4 regression)

For $f = g = \sin(u)$, we have $f \cdot g = \sin^2(u) = (1 - \cos(2u))/2$ with exact derivative supnorms

$$\|(\sin^2)^{(m)}\|_\infty = \begin{cases} 1, & m = 0 \\ 2^{m-1}, & m \geq 1. \end{cases}$$

The exact-Leibniz formula (2) gives, for $\|\sin^{(k)}\|_\infty = 1$ at all k, $\sum_k \binom{m}{k} \cdot 1 \cdot 1 = 2^m$, which is at most $2\times$ the truth for $m \geq 1$ and tight at $m = 0$. The test verifies $\texttt{sup_bounds}_{fg}[m{+}1] \geq \|(\sin^2)^{(m)}\|_\infty$ for every $m = 0, \ldots, M_{\text{order}} + 1$ and every $M_{\text{order}} \in \{2, 4, 6\}$.

Result: 18/18 assertions pass.

4.9 Test T2.9: `inv_series` `sup_bounds` responds to input frequency (audit C1 regression)

For $f(u) = 2 + \cos(Nu)$ with $\|f^{(k)}\|_\infty = N^k$ ($k \geq 1$), the Faà di Bruno bound on $\|(1/f)^{(M+1)}\|_\infty$ should scale roughly as N^{M+1}. At $M_{\text{order}} = 2$, the test records:

$$N = 1 : \texttt{sup_bounds}[\text{end}] = 13.0, \quad N = 3 : 351.0, \quad N = 10 : 13\,000.$$

The growth $13 \to 13\,000$ for $N = 1 \to 10$ matches the expected $N^3 = 1000\times$ scaling (Faà di Bruno single-block term $\phi^{(1)}(f) \cdot f^{(M+1)}$). The previous formula returned a constant ≈ 162 independent of N — unsound by orders of magnitude.

Result: 3/3 assertions pass.

4.10 Test T2.10: `sqrt_series` `sup_bounds` responds to input frequency (audit C2 regression)

Same setup as T2.9 but for `sqrt_series`. Recorded values:

$$N = 1 : \texttt{sup_bounds}[\text{end}] = 1.625, \quad N = 3 : 43.875, \quad N = 10 : 1\,625.$$

The previous formula was dimensionally inconsistent and ignored f's derivative bounds entirely; the new sup_bounds vector correctly grows with N.

Result: 3/3 assertions pass.

5 Summary

Test	Description
T2.1	Taylor expansion contains true $\sin(u)$ at 200 points
T2.2	`sup_bound` $\geq \|\sin\|_\infty = 1$ for orders 2,4,6
T2.3	$D_{\text{lower}} > 0.88$ for $y = 0.1\cos u$ (true $D_{\min} = 0.9$)
T2.4	`sup_bound` decreases with M_{order}
T2.5	`deriv_series`(cos) $= -\sin$ at all grid points
T2.6	`ts_sin` $\times$ `ts_cos` $=$ `ts`$_{\sin(2u)/2}$
T2.7	`inv_series`: $1/(2+\cos u)$ at 32 grid points
T2.8	Product `sup_bounds` $\geq \|(\sin^2)^{(m)}\|_\infty$ (audit C4)
T2.9	`inv_series` `sup_bounds` scales with N (audit C1)
T2.10	`sqrt_series` `sup_bounds` scales with N (audit C2)
T2.11	`_faa_di_bruno_derivs` against analytic compositions $(g \circ C)$ with $g \in \{x, x^2, \exp\}$, $C \in \{z+1, z, z^2 +$
T2.12	`dmin_bound` throws on degenerate y ($D \not> 0$)
T2.13	$M_{\text{order}} = 0$ edge case (smallest legal): `sup_bound`, $*$, `deriv_series` reject, `inv_series`
T2.14	`trigpoly_to_taylor`: derivs + sup_bounds vs. analytic sin at 32 grid points $\times$ 5 derivative levels
Total	

Table 1: M2 test summary. All 581 assertions pass at 128-bit Arb precision. Tests T2.8–T2.10 guard against the audit findings on the old single-scalar `F_bound` formulas. T2.11–T2.14 (post-implementation re-audit) add direct coverage of `_faa_di_bruno_derivs`, the `dmin_bound` throw path, the $M_{\text{order}} = 0$ edge case, and `trigpoly_to_taylor`.

Milestone M2 is complete. The `TaylorSeries.jl` module provides:

- A certified Taylor-series representation of smooth functions at grid points, with a *vector* of supnorm bounds covering every derivative order from 0 to $M_{\text{order}} + 1$.
- Rigorous `sup_bound` and `inf_bound` using the remainder formula (1).
- Arithmetic (add, multiply, scale, differentiate) with certified `sup_bounds` propagation via *exact* Leibniz term-by-term.
- Recursive `inv_series` and `sqrt_series` with `sup_bounds` via Faà di Bruno (`ArbSeries` composition).
- `dmin_bound`(y): a certified lower bound on $D = \sqrt{y_u^2 + (1 + \hat{H}[y_u])^2}$ using only exact `TrigPoly` derivatives (from M1) and the Taylor remainder formula.

The next milestone will use `TaylorSeries` to bound all terms in the Babenko residual operator between grid points, completing the certified $\|R\|_\infty \leq \varepsilon$ estimate.